IT110 Lab: Floating Point Arithmetic

IEEE 754 Standard

Instructor

Due: Feb 6th, 11:59 pm

Key Concepts to Review

Before we begin, ensure you understand these three fundamentals:

1. **Normalization**: Moving the decimal point until exactly one '1' is to the left of it.
   Example: 1.011×2^3

Key Concepts to Review

Before we begin, ensure you understand these three fundamentals:

1. **Normalization**: Moving the decimal point until exactly one '1' is to the left of it.
   Example: 1.011×2^3
2. **Bias (127)**: In 32-bit single-precision, you must add 127 to the actual exponent. This allows us to represent negative exponents without using a sign bit in the exponent field.

Key Concepts to Review

Before we begin, ensure you understand these three fundamentals:

1. **Normalization**: Moving the decimal point until exactly one '1' is to the left of it.
   Example: 1.011×2^3
2. **Bias (127)**: In 32-bit single-precision, you must add 127 to the actual exponent. This allows us to represent negative exponents without using a sign bit in the exponent field.
3. **Hidden Bit**: The leading "1" before the decimal point in normalized binary is *implied*. You do not store it in the mantissa.

The 32-bit IEEE 754 Structure

The 3 Fields

1. **Sign Bit (1 bit)**:
   $0 =$ Positive $(+)$, $1 =$ Negative (-)
2. **Exponent (8 bits)**:
   Value $=$ Actual Exponent $+ 127$
3. **Mantissa (23 bits)**:
   The fractional part (after the normalized 1).

Format: $\boxed{\text{S}}\ \boxed{\text{Exponent (8)}}\ \boxed{\text{Mantissa (23)}}$

Part 1: Decimal to IEEE 754

Goal: Convert decimal numbers into the 32-bit binary format.

Question 1a: Convert 195

Step 1: Convert Integer to Binary

- $195 = 128 + 64 + 2 + 1$
- Binary: **`11000011`**

Question 1a: Convert 195

Step 1: Convert Integer to Binary

- $195 = 128 + 64 + 2 + 1$
- Binary: **`11000011`**

Step 2: Normalize

- Shift the decimal point left until it is behind the first '1'.
- Original: 11000011.0
- Shift Left: 7 places
- **Normalized:** 1.1000011×2^7

Question 1a: Convert 195

Step 1: Convert Integer to Binary

- $195 = 128 + 64 + 2 + 1$
- Binary: **11000011**

Step 2: Normalize

- Shift the decimal point left until it is behind the first '1'.
- Original: 11000011.0
- Shift Left: 7 places
- **Normalized:** 1.1000011×2^7

Step 3: Biased Exponent

- Exponent is 7. Add Bias (127).
- $7 + 127 = 134 \rightarrow$ **10000110**

Question 1a: Assembly

Step 4: The Mantissa

- Take the bits *after* the decimal point from Step 2.
- Normalized: $1.\underline{1000011}$
- Pad with zeros to reach 23 bits.
- Field: **`10000110000000000000000`**

Question 1a: Assembly

Step 4: The Mantissa

- Take the bits *after* the decimal point from Step 2.
- Normalized: $1.\underline{1000011}$
- Pad with zeros to reach 23 bits.
- Field: **10000110000000000000000**

Final Answer

Sign: 0 $(+)$
Result: 0 10000110 10000110000000000000000

Question 1b: Convert 0.717700

Step 1: Fraction to Binary Multiply by 2. If ≥ 1, record '1', else '0'.

$$0.7177 \times 2 = 1.4354 \rightarrow \mathbf{1}$$
$$0.4354 \times 2 = 0.8708 \rightarrow \mathbf{0}$$
$$0.8708 \times 2 = 1.7416 \rightarrow \mathbf{1}$$
$$0.7416 \times 2 = 1.4832 \rightarrow \mathbf{1}$$
$$0.4832 \times 2 = 0.9664 \rightarrow \mathbf{0}$$
$$0.9664 \times 2 = 1.9328 \rightarrow \mathbf{1}$$

Binary Approx: 0.101101...

Question 1b: Normalization & Bias

Step 2: Normalize

- Shift right to get the first '1' before the decimal.
- $0.101101... \rightarrow 1.01101... \times 2^{-1}$

Question 1b: Normalization & Bias

Step 2: Normalize

- Shift right to get the first '1' before the decimal.
- 0.101101... $\rightarrow$ 1.01101... $\times 2^{-1}$

Step 3: Biased Exponent

- Exponent: -1
- Bias: $-1 + 127 = 126$
- Field: **`01111110`**

Question 1b: Normalization & Bias

Step 2: Normalize

- Shift right to get the first '1' before the decimal.
- $0.101101... \rightarrow 1.01101... \times 2^{-1}$

Step 3: Biased Exponent

- Exponent: -1
- Bias: $-1 + 127 = 126$
- Field: **`01111110`**

Final Answer

Sign: 0
Result: **`0 01111110 0110111...`**

Part 2: IEEE 754 to Decimal

Goal: Decode the binary string back into a decimal number.

Remember: Identify the Sign, Exponent, and Mantissa first.

Q2a: `0 10000101 10111...`

Step 1: Decode Fields

- **Sign:** 0 (Positive)
- **Exponent:** **`10000101`** $= 128 + 4 + 1 = 133$
- **Mantissa:** **`10111...`**

Q2a: `0 10000101 10111...`

Step 1: Decode Fields

- **Sign:** 0 (Positive)
- **Exponent:** `10000101` $= 128 + 4 + 1 = 133$
- **Mantissa:** `10111...`

Step 2: Unbias Exponent

- $133 - 127 = 6$
- Multiplier is 2^6

Q2a: `0 10000101 10111...`

Step 1: Decode Fields

- **Sign:** 0 (Positive)
- **Exponent: `10000101`** $= 128 + 4 + 1 = 133$
- **Mantissa: `10111...`**

Step 2: Unbias Exponent

- $133 - 127 = 6$
- Multiplier is 2^6

Step 3: Convert

- Restore hidden bit: 1.1011100...
- Shift decimal right 6 places: 1101110.00...
- Integer: $64 + 32 + 8 + 4 + 2 = 110$
- **Value:** ≈ 110.00195

Q2b: `1 01110100 00001...`

Step 1: Decode Fields

- **Sign:** 1 (Negative!)
- **Exponent:** `01110100` $= 64 + 32 + 16 + 4 = 116$

Q2b: `1 01110100 00001...`

Step 1: Decode Fields

- **Sign:** 1 (Negative!)
- **Exponent:** `01110100` $= 64 + 32 + 16 + 4 = 116$

Step 2: Unbias Exponent

- $116 - 127 = -11$
- Multiplier is 2^{-11}

Q2b: `1 01110100 00001...`

Step 1: Decode Fields

- **Sign:** 1 (Negative!)
- **Exponent:** `01110100` $= 64 + 32 + 16 + 4 = 116$

Step 2: Unbias Exponent

- $116 - 127 = -11$
- Multiplier is 2^{-11}

Final Result

Equation: $-1.00001... \times 2^{-11}$
Decimal: $\approx -4.88 \times 10^{-4}$

Q2c: `0 10010011 11000...`

Step 1: Decode Fields

- **Sign:** $0\ (+)$
- **Exponent:** `10010011` $= 128 + 16 + 2 + 1 = 147$
- **Mantissa:** `11000...`

Q2c: `0 10010011 11000...`

Step 1: Decode Fields

- **Sign:** $0\ (+)$
- **Exponent:** **`10010011`** $= 128 + 16 + 2 + 1 = 147$
- **Mantissa:** **`11000...`**

Step 2: Unbias Exponent

- $147 - 127 = 20$
- Multiplier is 2^{20}

Q2c: `0 10010011 11000...`

Step 1: Decode Fields

- **Sign:** 0 (+)
- **Exponent: `10010011`** $= 128 + 16 + 2 + 1 = 147$
- **Mantissa: `11000...`**

Step 2: Unbias Exponent

- $147 - 127 = 20$
- Multiplier is 2^{20}

Step 3: Denormalize

- $1.11000... \times 2^{20}$
- Shift right 20 places: **`111000000100000100000`**

Value: $1,837,088$

Summary

Don't forget to submit your lab by Feb 6th, 11:59 pm!